

Tugas Kecil 2 IF2211 Strategi Algoritma
Semester II Tahun 2025/2026
Voxelization Objek



Disusun oleh :

Nathan Edward Christofer Marpaung, Richard Samuel Simanullang
13524062, 13525112

Program Studi Teknik Informatika
Sekolah Teknik dan Informatika
Institut Teknologi Bandung

2024

I. *Voxelization*

Voxelization adalah proses untuk menyederhanakan suatu model objek yang terdiri dari berbagai mesh menjadi objek yang disusun oleh kubus-kubus yang uniform dengan volume yang sama (voxels). Proses ini dilakukan dengan struktur data Octree, struktur pohon yang memiliki 8 anak pohon atau daun. Octree digunakan untuk membagi kubus uniform tersebut menjadi 8 anak kubus berbeda yang dapat dipotong lagi. Proses ini dilakukan berulang kali hingga mencapai kedalaman yang diinginkan dan menyerupai objek awal yang disusun oleh kubus voxels tersebut

II. *Algoritma Divide and Conquer*

Algoritma *Divide and Conquer* adalah strategi algoritma yang membantu menyelesaikan berbagai masalah kompleks dengan membagi permasalahan utama menjadi beberapa sub-masalah. Algoritma akan membagi permasalahan menjadi sub-masalah secara rekursif hingga mencapai basis, dimana setiap sub-masalah akan diselesaikan secara tersendiri. Hasil solusi dari penyelesaian sub-masalah kemudian digabungkan kembali hingga menjadi solusi utama dari permasalahan utama tersebut.

Dalam konteks tugas ini, Algoritma *Divide and Conquer* digunakan untuk membantu proses *Voxelization* dari sebuah objek. *Divide and Conquer* digunakan untuk membagi kubus menjadi 8 sub-kubus untuk dipecahkan. Setiap sub-kubus yang berpotongan dengan mesh objek akan dibagi kembali menjadi sub-kubus, sedangkan sub-kubus yang tidak berpotongan akan dihilangkan. Hal ini dilakukan secara rekursif hingga mencapai kedalaman yang diinginkan. *Divide and Conquer* akan digunakan untuk membantu pembagian Octree berdasarkan hasil berpotongan. Pengecekan berpotongan akan dilakukan melalui *Axis-Aligned Bounding Boxes (AABB) Collision Detection* dan *Separating Axis Theorem (SAT)*

III. *Axis-Aligned Bounding Boxes Collision Detection*

Axis-Aligned Bounding Boxes atau AABB adalah salah satu teknik algoritma untuk mengecek perpotongan antara dua benda. Setiap objek akan dianggap sebagai satu balok yang memiliki nilai panjang, lebar, dan tinggi maksimum sesuai dengan objek tersebut. Algoritma akan menggunakan balok-balok tersebut untuk mendeteksi apakah ada perpotongan antara dua objek

atau tidak. Perpotongan akan dideteksi dengan mengecek apakah ada titik yang berada di dalam kubus yang membungkus objek tersebut atau tidak.

Tugas ini menggunakan AABB sebagai salah satu teknik *collision detection*, yaitu *AABB vs AABB 3D Collision Detection*. Deteksi akan diperiksa antara 2 objek AABB, yaitu voxel dengan sub-mesh dari objek awal, dimana voxel akan dibagi lagi menjadi 8 sub-voxel jika *AABB Collision* terdeteksi dan salah satu titik dari voxel berpotongan atau berada di dalam AABB dari sub-mesh objek awal.

IV. Separating Axis Theorem (SAT)

Metode sebelumnya menggunakan AABB vs AABB dengan membungkus segitiga yang ada dengan kubus baru. Hal ini menyebabkan banyaknya node yang dianggap intersect padahal hanya kubus pembungkus segitiganya yang intersect, bukan segitiganya. Dengan banyaknya node yang ada, mulai depth = 9 ukurannya cukup besar sehingga sulit untuk dibaca reader.

Untuk mengatasi masalah tersebut, diambil pendekatan Separating Axis Theorem 3D yang biasa disebut SAT. Separating Axis Theorem mempunyai ide utama dimana jika tidak dapat ditemukan 1 sumbu yang membuat 2 objek tidak overlap, maka dipastikan 2 objek tersebut intersect. Biasanya kalau 2D yang dicek itu adalah sumbu utama dan proyeksi min/max objeknya. Untuk 3D, kita buruh mengecek sumbu utama, sumbu normal segitiga, serta hasil cross product sudut dengan sisi objeknya.

Langkah selanjutnya, dihitung tiga vektor sisi segitiga yaitu $e0 = v1 - v0$, $e1 = v2 - v1$, dan $e2 = v0 - v2$. Ketiga vektor sisi ini digunakan untuk membentuk 9 sumbu pengujian dari cross product antara setiap sisi segitiga dengan setiap sumbu utama koordinat (X, Y, Z). Untuk setiap sumbu pengujian tersebut, ketiga vertex segitiga diproyeksikan ke sumbu itu dan dibandingkan dengan jangkauan maksimum kotak (r) yang dihitung dari halfSize.

Kemudian diuji 3 sumbu utama (X, Y, Z). Ini setara dengan pengecekan AABB biasa — jika seluruh segitiga berada di luar batas kotak di salah satu sumbu, maka tidak ada intersect.

Terakhir, diuji sumbu normal bidang segitiga yang dihitung dari cross product $e_0 \times e_1$. Kotak diproyeksikan ke arah normal ini dengan menghitung seberapa jauh kotak bisa "menjangkau" ke arah tersebut (r). Jika jarak bidang segitiga dari origin (d) lebih jauh dari jangkauan kotak, berarti kotak tidak menembus bidang segitiga sama sekali dan fungsi kembali mengembalikan false.

V. Pembahasan Kode

Strategi Algoritma diimplementasikan menggunakan bahasa C++. Beberapa class, struktur data, dan metode (Point, Triangle, AABB, Octree) diimplementasikan di dalam kode untuk membantu berjalannya program.

1. Tahap awal

Input file obj akan diproses dan dibagi menjadi 2 vektor yang berisi vertice dan face dari file obj, dimana vertice disusun oleh struktur data Point yang berisi komponen axis x,y, dan z. Kedua vektor akan digunakan untuk diproses kembali menjadi satu vektor segitiga yang berisi ketiga vertice dari setiap face segitiga.

```
ALGORITMA parseOBJ(filepath)
```

```
INPUT: filepath
```

```
OUTPUT: vertices, faces, triangles
```

```
Buka file filepath
```

```
Jika file gagal dibuka
```

```
    Tampilkan error
```

```
    Hentikan program
```

```
Inisialisasi vertices sebagai vektor kosong
```

```
Inisialisasi faces sebagai vektor kosong
```

```
Inisialisasi triangles sebagai vektor kosong
```

```
Untuk setiap baris pada file
```

```
    Jika baris diawali 'v'
```

```
        Baca x, y, z
```

```
        Tambahkan Point(x, y, z) ke vertices
```

```
    Else if baris diawali 'f'
```

```
        Baca a, b, c
```

```
        Tambahkan Face(a, b, c) ke faces
```

```
    Endif
```

```
Endfor
```

```
Untuk setiap face dalam faces
```

```
    p1 = vertices[face.a - 1]
```

```
    p2 = vertices[face.b - 1]
```

```
    p3 = vertices[face.c - 1]
```

```
    Tambahkan Triangle(p1, p2, p3) ke triangles
```

```
Endfor
```

```
Return vertices, faces, triangles
```

Nilai minimum dan maksimum titik di setiap axis kemudian digunakan untuk membuat bounding box utama yang membungkus objek. Dengan adanya bounding box utama, nilai bounding box dan list triangle dari objek digunakan untuk membentuk Octree utama.

```
ALGORITMA buildRootOctree(vertices, triangles)
```

```
INPUT: vertices, triangles
```

```
OUTPUT: rootBox, rootOctree
```

```
minX = vertices[0].x
```

```
minY = vertices[0].y
```

```
minZ = vertices[0].z
```

```
maxX = vertices[0].x
```

```
maxY = vertices[0].y
```

```
maxZ = vertices[0].z
```

```
Untuk setiap point p dalam vertices
```

```
    minX = minimum(minX, p.x)
```

```
    minY = minimum(minY, p.y)
```

```
    minZ = minimum(minZ, p.z)
```

```
    maxX = maksimum(maxX, p.x)
```

```
    maxY = maksimum(maxY, p.y)
```

```
    maxZ = maksimum(maxZ, p.z)
```

```
Endfor
```

```
rootBox.min = Point(minX, minY, minZ)
```

```
rootBox.max = Point(maxX, maxY, maxZ)
```

```
Buat rootOctree dengan:
```

```
    bounds = rootBox
```

```
    triangles = triangles
```

```
    children = kosong
```

```
Return rootBox, rootOctree
```

Algoritma Divide and Conquer kemudian diterapkan dengan membagi bounding box tersebut menjadi 8 sub-kubus voxel sebagai 8 anak dari Octree utama. Setiap sub-kubus merupakan tipe data Octree juga, sehingga dapat membagi dirinya menjadi 8 sub-kubus/voxel yang lebih kecil lagi.

Pada setiap pembuatan anak Octree, sebuah bounding box voxel akan dibentuk dengan titik center bounding box utama sebagai salah satu vertice dari bounding box. Hasil bounding box kemudian akan digunakan untuk memeriksa apakah terjadi berpotongan antara bounding box voxel dengan bounding box dari segitiga. Jika terjadi berpotongan, maka setiap segitiga yang berpotongan dengan bounding box voxel akan disimpan dalam sebuah vector childTriangle yang akan digunakan kembali untuk mengecek sub-voxel selanjutnya. Algoritma ini kemudian akan dijalankan kembali secara rekursif pada node anak Octree yang berpotongan dengan objek, menggunakan childTriangle yang sudah disimpan sebagai parameter untuk mengecek lagi perpotongan sub-voxel seterusnya. Algoritma ini akan berjalan hingga mencapai kedalaman yang diinginkan. Jika tidak ada terjadi perpotongan dengan triangle face dari objek, maka node anak tersebut akan diisi dengan nullptr dan di pruning.

Pada setiap kedalaman, algoritma akan menyimpan jumlah node yang berhasil berpotongan dengan objek dan jumlah node yang tidak berpotongan dan di pruning. Setiap voxel yang berhasil diciptakan pada depth terakhir akan dihitung pada variabel voxelCnt. Waktu eksekusi dari pencarian voxel dan pembuatan Octree juga dihitung, dengan program mencatat waktu dimulai dari pembuatan Octree awal dengan root bounding box hingga seluruh bagian algoritma selesai berjalan.

Hasil pembuatan Octree dan perhitungan node, pruned node, dan jumlah Voxel akan digunakan untuk menampilkan hasil statistika dari pencarian voxel dengan Octree tersebut. Jumlah node yang dibentuk, jumlah node yang di pruning, dan jumlah voxel yang ditampilkan dalam CLI program saat dijalankan. Waktu yang digunakan untuk menjalankan program juga ditampilkan di dalam CLI.

Hasil Octree yang disimpan juga digunakan untuk membuat file .obj akhir yang sudah di voxelize menggunakan metode writeVoxels. Metode akan menelusuri setiap noda yang memiliki anak hingga mencapai kedalaman maksimum. Pada kedalaman maksimum, setiap 8 vertice dari voxel yang dibentuk akan disimpan dalam file output. Kemudian, face dari voxel akan di output ke dalam file .obj akhir secara berurutan menggunakan indeks dari vertice tersebut. Metode ini berjalan secara rekursif hingga mencapai semua voxel di kedalaman akhir.

Pseudocode

Hasil pseudocode akhir dari program utama adalah:

```
procedure buildOctree(bbox, triangles, currDepth, maxDepth,
                    voxelCnt, nodesPerDepth, prunedPerDepth, method)

    if triangles kosong then
        prunedPerDepth[currDepth] = prunedPerDepth[currDepth] + 1
        return null
    end if

    if currDepth = maxDepth then
        nodesPerDepth[currDepth] = nodesPerDepth[currDepth] + 1
        voxelCnt = voxelCnt + 1
        buat node voxel dengan bbox
        return node
    end if

    nodesPerDepth[currDepth] = nodesPerDepth[currDepth] + 1
    buat node internal dengan bbox

    childBoxes = bagi bbox menjadi 8 sub-box
    for setiap childBox dalam childBoxes do
        childTriangles = kosong
        for setiap triangle dalam triangles do
            if method = SAT then
                intersect = childBox.isIntersectSAT(triangle)
            else
                intersect = childBox.isIntersect(triangle)
            end if
            if intersect then
                masukkan triangle ke childTriangles
            end if
        end for
    end for
```

```

childNode = buildOctree(childBox, childTriangles, currDepth + 1,
                        maxDepth, voxelCnt, nodesPerDepth,
                        prunedPerDepth, method)

    simpan childNode sebagai anak node
end for

return node
end procedure

```

VI. Hasil

1. Cow.obj

Statistik	Output gambar
Banyaknya voxel : 49071 Banyaknya vertex : 392568 Banyaknya faces : 588852 --- Statistik Node Octree Terbentuk --- 1 : 1 2 : 8 3 : 20 4 : 96 5 : 372 6 : 1764 7 : 8797 8 : 49071 --- Statistik Node Tidak Perlu Ditelusuri --- 1 : 0 2 : 0 3 : 44 4 : 64 5 : 396	

6 : 1212 7 : 5315 8 : 21305 Kedalaman octree : 8 Waktu eksekusi : 26.4705 ms	
---	--

2. Line.obj

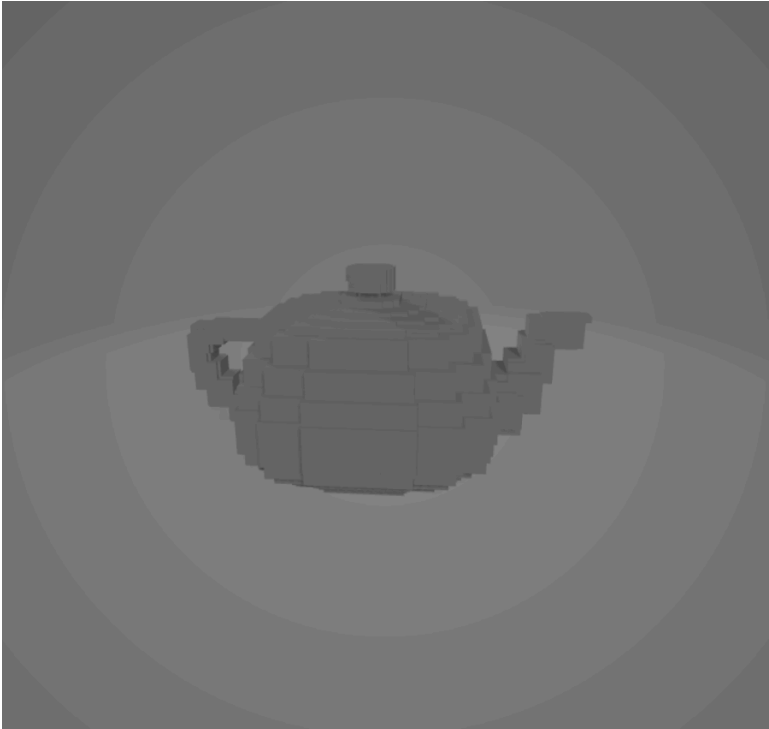
Statistik	Output gambar
Banyaknya voxel : 359424 Banyaknya vertex : 2875392 Banyaknya faces : 4313088 --- Statistik Node Octree Terbentuk --- 1 : 1 2 : 8 3 : 32 4 : 128 5 : 896 6 : 7168 7 : 48384 8 : 359424 --- Statistik Node Tidak Perlu Ditelusuri --- 1 : 0 2 : 0 3 : 32 4 : 128 5 : 128 6 : 0 7 : 8960 8 : 27648 Kedalaman octree : 8 Waktu eksekusi : 108.039 ms	

3. Pumpkin.obj

Statistik	Output gambar
Banyaknya voxel : 190455 Banyaknya vertex : 1523640 Banyaknya faces : 2285460 --- Statistik Node Octree Terbentuk --- 1 : 1 2 : 8 3 : 56 4 : 300 5 : 1345 6 : 6421 7 : 33134 8 : 190455 --- Statistik Node Tidak Perlu Ditelusuri --- 1 : 0 2 : 0 3 : 8 4 : 148 5 : 1055 6 : 4339 7 : 18234 8 : 74617 Kedalaman octree : 8 Waktu eksekusi : 95.7306 ms	

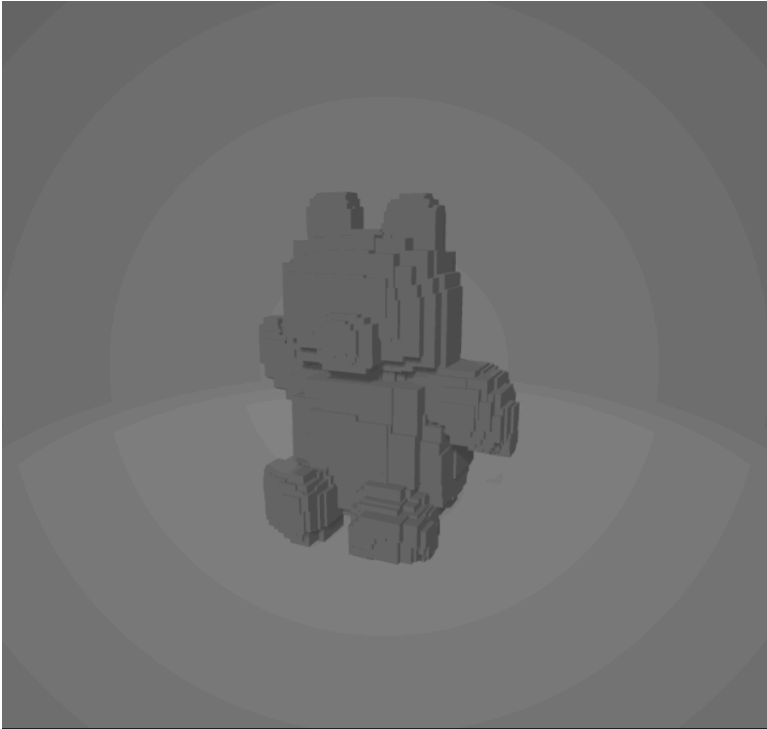
4. Teapot.obj

Statistik	Output gambar
-----------	---------------

Banyaknya voxel : 114489 Banyaknya vertex : 915912 Banyaknya faces : 1373868 --- Statistik Node Octree Terbentuk --- 1 : 1 2 : 8 3 : 26 4 : 128 5 : 573 6 : 3044 7 : 17892 8 : 114489 --- Statistik Node Tidak Perlu Ditelusuri --- 1 : 0 2 : 0 3 : 38 4 : 80 5 : 451 6 : 1540 7 : 6460 8 : 28647 Kedalaman octree : 8 Waktu eksekusi : 38.3085 ms	
--	--

5. Teddy.obj

Statistik	Output Gambar
-----------	---------------

Banyaknya voxel : 153919 Banyaknya vertex : 1231352 Banyaknya faces : 1847028 --- Statistik Node Octree Terbentuk --- 1 : 1 2 : 8 3 : 39 4 : 162 5 : 842 6 : 4303 7 : 24359 8 : 153919 --- Statistik Node Tidak Perlu Ditelusuri --- 1 : 0 2 : 0 3 : 25 4 : 150 5 : 454 6 : 2433 7 : 10065 8 : 40953 Kedalaman octree : 8 Waktu eksekusi : 48.1455 ms	
---	--

6. Cow.obj (depth 10 dengan SAT)

Statistik	Output Gambar
-----------	---------------

Banyaknya voxel : 369094 Banyaknya vertex : 2952752 Banyaknya faces : 4429128 --- Statistik Node Octree Terbentuk --- 1 : 1 2 : 8 3 : 20 4 : 88 5 : 332 6 : 1362 7 : 5627 8 : 22864 9 : 92099 10 : 369094 --- Statistik Node Tidak Perlu Ditelusuri --- 1 : 0 2 : 0 3 : 44 4 : 72 5 : 372 6 : 1294 7 : 5269 8 : 22152 9 : 90813 10 : 367698 Kedalaman octree : 10 Waktu eksekusi : 160.659 ms	
---	--

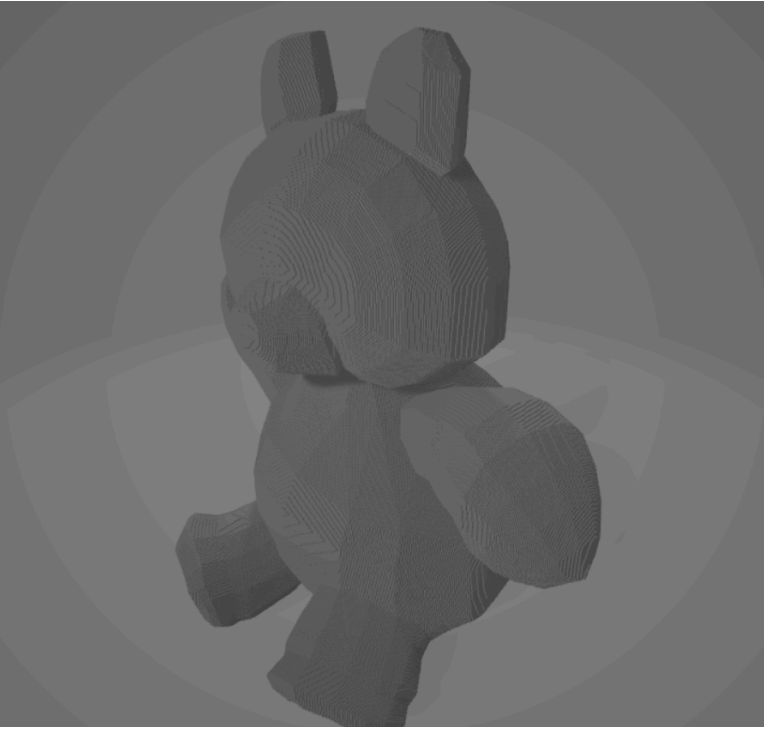
7. Pumpkin.obj (depth 10 dengan SAT)

Statistik	Output Gambar
-----------	---------------

Banyaknya voxel : 1196544 Banyaknya vertex : 9572352 Banyaknya faces : 14358528 --- Statistik Node Octree Terbentuk --- 1 : 1 2 : 8 3 : 56 4 : 277 5 : 1123 6 : 4626 7 : 18637 8 : 74750 9 : 299111 10 : 1196544 --- Statistik Node Tidak Perlu Ditelusuri --- 1 : 0 2 : 0 3 : 8 4 : 171 5 : 1093 6 : 4358 7 : 18371 8 : 74346 9 : 298889 10 : 1196344 Kedalaman octree : 10 Waktu eksekusi : 464.998 ms	
--	--

8. Teddy.obj (depth 10 dengan SAT)

Statistik	Output Gambar
-----------	---------------

Banyaknya voxel : 702439 Banyaknya vertex : 5619512 Banyaknya faces : 8429268 --- Statistik Node Octree Terbentuk --- 1 : 1 2 : 8 3 : 35 4 : 147 5 : 655 6 : 2686 7 : 10881 8 : 43735 9 : 175409 10 : 702439 --- Statistik Node Tidak Perlu Ditelusuri --- 1 : 0 2 : 0 3 : 29 4 : 133 5 : 521 6 : 2554 7 : 10607 8 : 43313 9 : 174471 10 : 700833 Kedalaman octree : 10 Waktu eksekusi : 248.101 ms	
---	--

VII. Lampiran

No	Poin	Ya	Tidak
1	Program berhasil di kompilasi tanpa kesalahan	✓	

2	Program berhasil dijalankan	✓	
3	Hasil konversi .obj terdiri dari kubus-kubus kecil	✓	
4	Program informasi-informasi atau report	✓	
5	Program dapat menerima masukan .obj, dan mengembalikan output	✓	
6	[Bonus] Program menggunakan <i>concurrency</i>		✓
7	[Bonus] Program dapat menampilkan visualisasi .obj		✓

Tugas ini disusun sepenuhnya tanpa bantuan kecerdasan buatan (Generative AI), melainkan hasil pemikiran dan analisis mandiri.



Nathan Edward Christofer Marpaung



Richard Samuel Simanullang

VIII. Referensi

- [1] T. Akenine-Möller, “Code by Tomas Akenine-Möller,” Lund University, [Online]. https://fileadmin.cs.lth.se/cs/Personal/Tomas_Akenine-Moller/code/. [diakses: Mar. 26, 2026].
- [2] P. T. Mikkelsen, “Sparse Solid Mesh Voxelization Tutorial,” Visual Computing Lab, Feb. 9, 2016. [Online]. <https://viscomp.alexandra.dk/index298d.html?p=3836>. [diakses: Mar. 26, 2026].
- [3] Skeffles, “How 2D Game Collision Works (Separating Axis Theorem),” YouTube, [Online video]. <https://www.youtube.com/watch?v=dn0hUgsok9M>. [diakses: Mar. 26, 2026].

IX. Pranala

https://github.com/Lloyd565/Tucil2_13524062_13524112